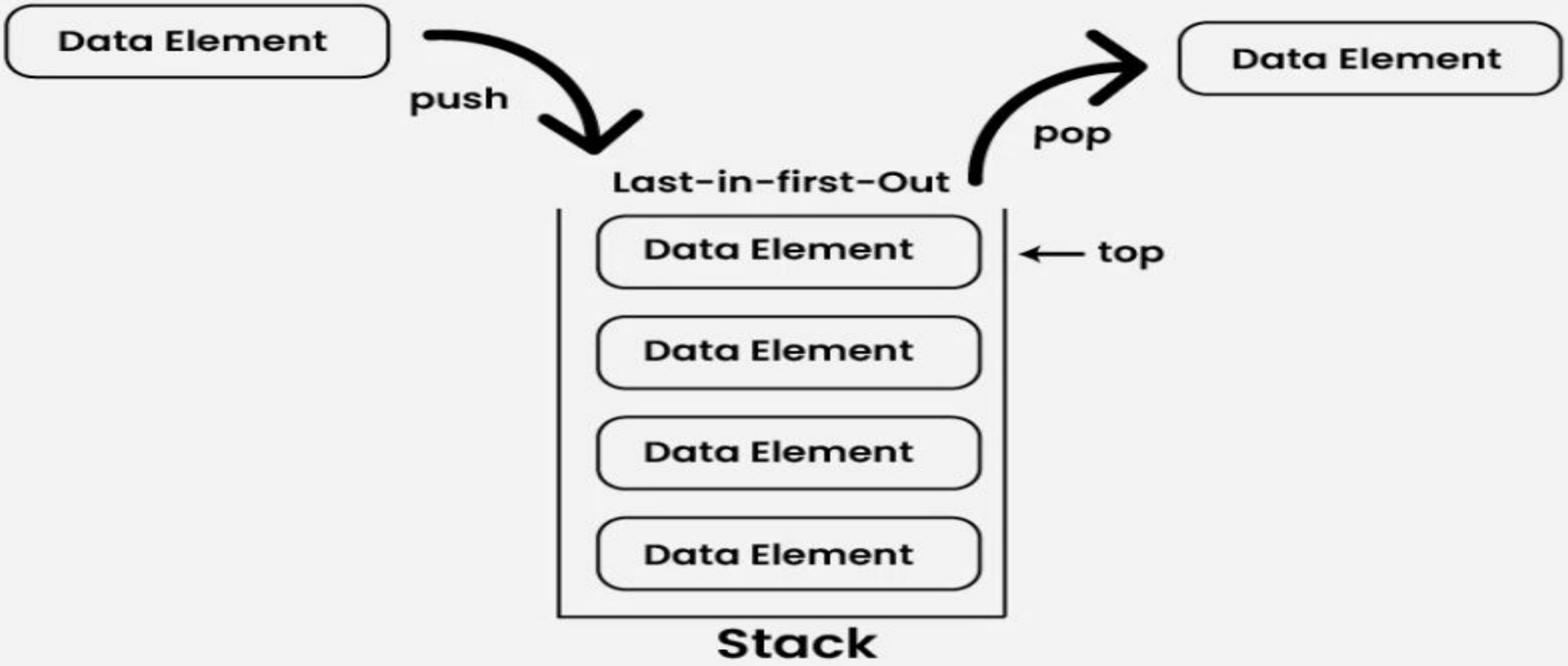
UNIT-2

Prepared By- Ms. Anjli
Asst. Prof.

What is Stack?

- A **Stack** is a linear data structure that follows a particular order in which the operations are performed.
- The order may be **LIFO(Last In First Out)** or **FILO(First In Last Out)**. **LIFO** implies that the element that is inserted last, comes out first and **FILO** implies that the element that is inserted first, comes out last.
- For Example- the last plate added is the first one to be removed.

Representation of Stack Data Structure



Types of stack

- **Fixed Size Stack** : As the name suggests, a fixed size stack has a fixed size and cannot grow or shrink dynamically. If the stack is full and an attempt is made to add an element to it, an overflow error occurs. If the stack is empty and an attempt is made to remove an element from it, an underflow error occurs.

- **Dynamic Size Stack** : A dynamic size stack can grow or shrink dynamically. When the stack is full, it automatically increases its size to accommodate the new element, and when the stack is empty, it decreases its size. This type of stack is implemented using a linked list, as it allows for easy resizing of the stack.

Basic Operations

- **push()** to insert an element into the stack
- **pop()** to remove an element from the stack
- **top()** Returns the top element of the stack.
- **isEmpty()** returns true if stack is empty else false.
- **isFull()** returns true if the stack is full else false.

To implement stack, we need to maintain reference to the top item.

Push Operation on Stack

- **ALGORITHM**

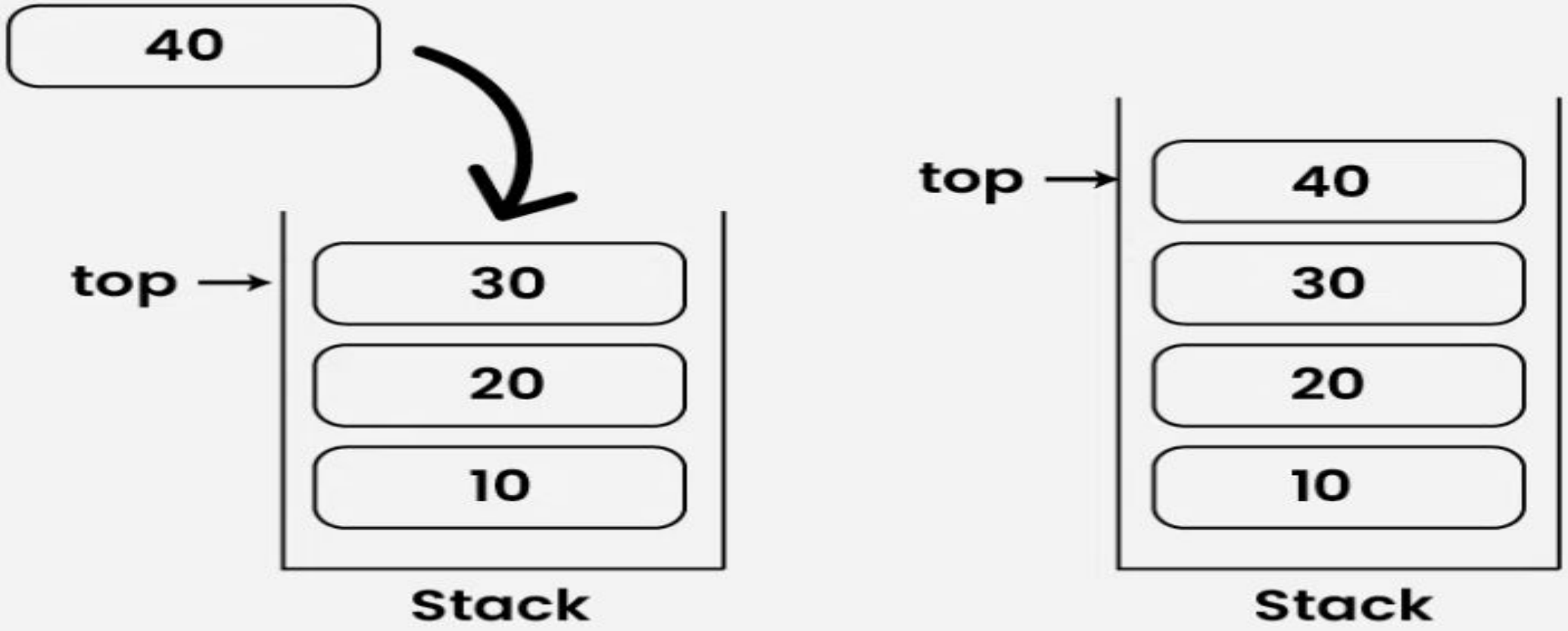
Before pushing the element to the stack, we check if the stack is **full** .

If the stack is full **(top == capacity-1)** , then **Stack Overflows** and we cannot insert the element to the stack.

Otherwise, we increment the value of top by 1 **(top = top + 1)** and the new value is inserted at **top position** .

The elements can be pushed into the stack till we reach the **capacity** of the stack.

Push Operation in Stack



```c
#include <stdio.h>
#include<conio.h>
int stack[5];  // Stack array of fixed size 5
int top = -1;  // Initialize top as -1 (empty stack)

void push(int value) {
    if (top == 4) {  // Check if the stack is full (MAX_SIZE - 1)
        printf("Stack Overflow! Cannot push %d\n", value);
        return;
    }
    top++;             // Increment top
    stack[top] = value;  // Insert value
    printf("Pushed %d into the stack.\n", value);
}

void display() {
    if (top == -1) {
        printf("Stack is empty!\n");
        return;
    }
    printf("Stack elements: ");
    for (int i = 0; i <= top; i++) {
        printf("%d ", stack[i]);
    }
    printf("\n");
}
```

```
int main() {
    push(10);
    push(20);
    push(30);
    push(40);
    push(50);
    push(60);  // This will cause stack overflow

    display(); // Show stack contents

    return 0;
}
```

```
Pushed 10 into the stack.
Pushed 20 into the stack.
Pushed 30 into the stack.
Pushed 40 into the stack.
Pushed 50 into the stack.
Stack Overflow! Cannot push 60
Stack elements: 10 20 30 40 50


...Program finished with exit code 0
Press ENTER to exit console.
```

Pop Operation in Stack

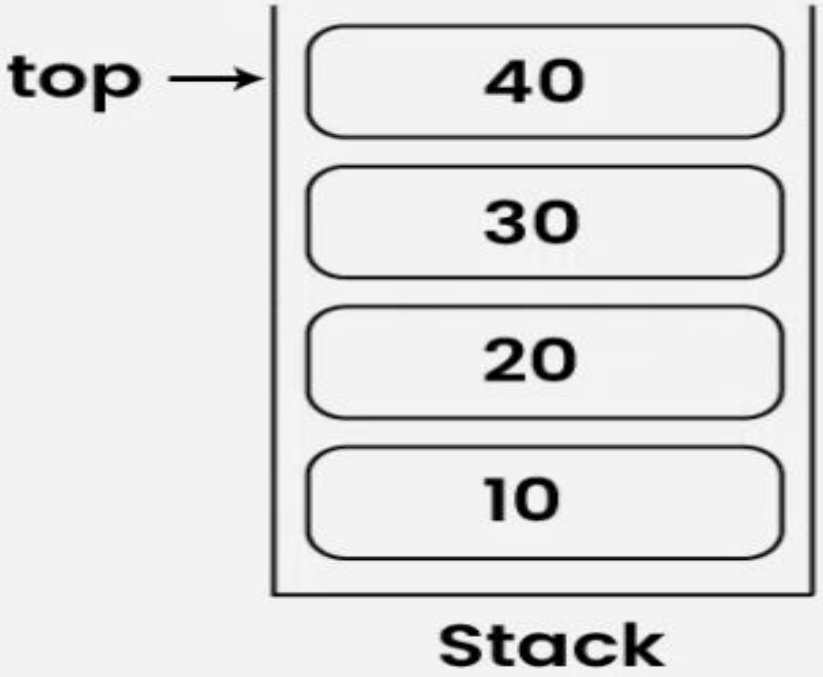
- **ALGORITHM**

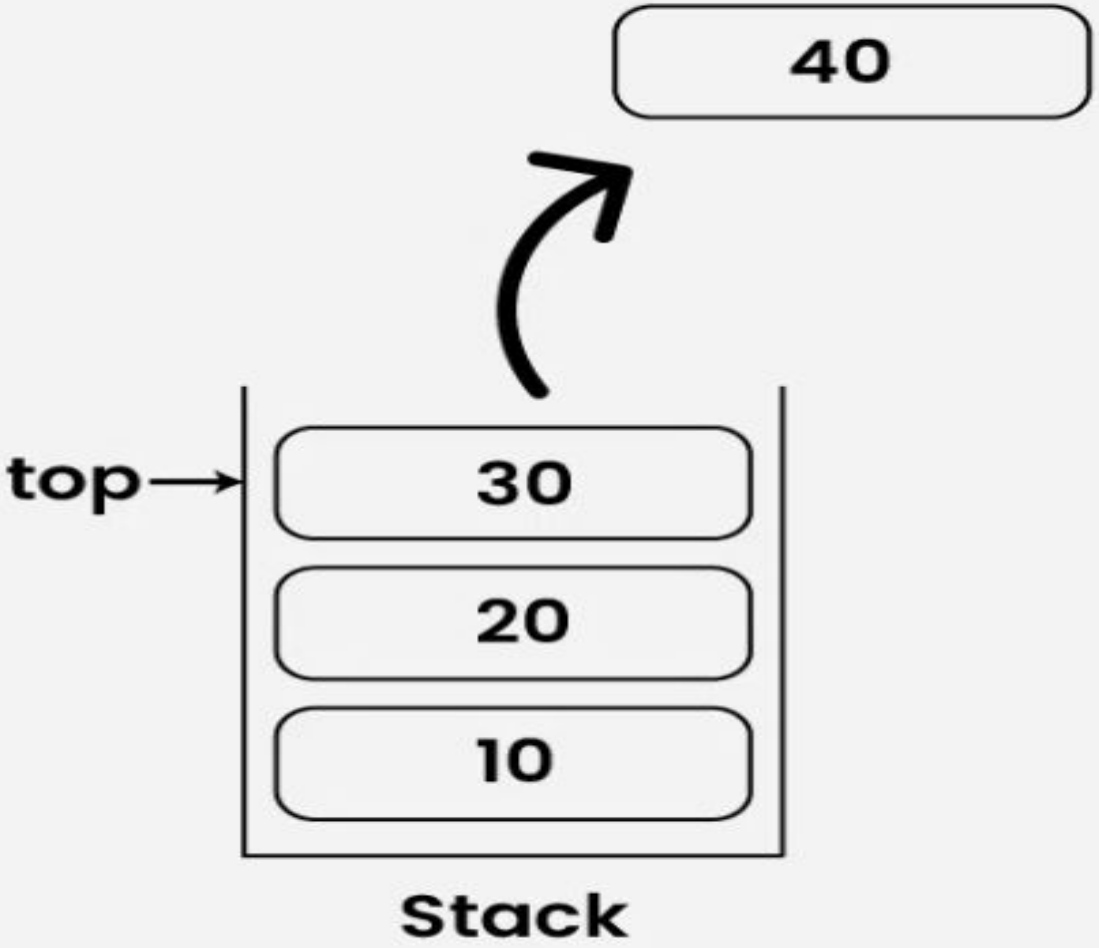
Before popping the element from the stack, we check if the stack is **empty** .

If the stack is empty (top == -1), then **Stack Underflows** and we cannot remove any element from the stack.

Otherwise, we store the value at top, decrement the value of top by 1 **(top = top – 1)** and return the stored top value.

Pop Operation in Stack





```c
#include <stdio.h>
#include<conio.h>
int stack[5];  // Stack array of fixed size 5
int top = -1;  // Initialize top as -1 (empty stack)

void push(int value) {
    if (top == 4) {  // Check if the stack is full
        printf("Stack Overflow! Cannot push %d\n", value);
        return;
    }
    top++;
    stack[top] = value;
    printf("Pushed %d into the stack.\n", value);
}

void pop() {
    if (top == -1) {  // Check if the stack is empty
        printf("Stack Underflow! Cannot pop.\n");
        return;
    }
    printf("Popped %d from the stack.\n", stack[top]);
    top--;  // Decrease top index
}
```

```c
void display() {
    if (top == -1) {
        printf("Stack is empty!\n");
        return;
    }
    printf("Stack elements: ");
    for (int i = 0; i <= top; i++) {
        printf("%d ", stack[i]);
    }
    printf("\n");
}

int main() {
    push(10);
    push(20);
    push(30);
    push(40);
    push(50);
    push(60);  // This will cause stack overflow
     display(); // Show stack contents
    pop();  // Remove top element
    pop();
    display(); // Show updated stack contents
    return 0;
}
```

```
Pushed 10 into the stack.
Pushed 20 into the stack.
Pushed 30 into the stack.
Pushed 40 into the stack.
Pushed 50 into the stack.
Stack Overflow! Cannot push 60
Stack elements: 10 20 30 40 50
Popped 50 from the stack.
Popped 40 from the stack.
Stack elements: 10 20 30


...Program finished with exit code 0
Press ENTER to exit console.
```

Top/ Peek Operation

- ALGORITHM
- Before returning the top element from the stack, we check if the stack is empty.
- If the stack is empty (top == -1), we simply print “Stack is empty”.
- Otherwise, we return the element stored at **index = top** .

```c
#include <stdio.h>
#include <conio.h>

int stack[5];  // Stack of fixed size 5
int top = -1;  // Top initialized to -1 (empty stack)

// Function to return the top element (Peek)
void peek() {
    if (top == -1) {
        printf("\nStack is empty! No top element.");
    } else {
        printf("\nTop element of stack: %d", stack[top]);
    }
}
```

```c
void main() {

    // Example: Pre-filling the stack with some values
    top = 2;
    stack[0] = 10;
    stack[1] = 20;
    stack[2] = 30;

    printf("\nStack initialized with some values.");

    peek();

    getch();
}
```

```
Stack initialized with some values.
Top element of stack: 30

...Program finished with exit code 255
Press ENTER to exit console.
```

isEmpty Operation

- ALGORITHM

[Returns true if the stack is empty, else false]

- Check for the value of **top** in stack.
- If **(top == -1)** , then the stack is **empty** so return **true** .
- Otherwise, the stack is not empty so return **false** .

```c
#include <stdio.h>
#include <conio.h>

int stack[5];  // Stack of fixed size 5
int top = -1;  // Top initialized to -1 (empty stack)

// Function to check if the stack is empty
void isEmpty() {
    if (top == -1) {
        printf("\nStack is empty.");
    } else {
        printf("\nStack is not empty.");
    }
}

void main() {

    printf("\nChecking if stack is empty initially:");
    isEmpty();  // Should print "Stack is empty."

    // Example: Pushing an element to the stack
    top = 0;
    stack[top] = 10;
```

```c
    printf("\n\nAfter pushing one element:");
    isEmpty();  // Should print "Stack is not empty."

    getch();
}
```

```
Checking if stack is empty initially:
Stack is empty.

After pushing one element:
Stack is not empty.

...Program finished with exit code 255
Press ENTER to exit console.
```

isFull Operation

- ALGORITHM

[Returns true if the stack is full, else false.]

- Check for the value of **top** in stack.
- If **(top == capacity-1),** then the stack is **full** so return **true** .
- Otherwise, the stack is not full so return **false.**

```c
#include <stdio.h>
#include <conio.h>

int stack[5];
int top = -1;

// Function to check if the stack is empty
void isEmpty() {
    if (top == -1) {
        printf("\nStack is empty.");
    } else {
        printf("\nStack is not empty.");
    }
}

// Function to check if the stack is full
void isFull() {
    if (top == 4) {  // Since stack size is 5 (index 0-4)
        printf("\nStack is full.");
    } else {
        printf("\nStack is not full.");
    }
}
```

```c
void main() {

    printf("\nChecking if stack is empty initially:");
    isEmpty();
    printf("\nChecking if stack is full initially:");
    isFull();

    // Example: Pushing elements to fill the stack
    top = 4;
    stack[0] = 10;
    stack[1] = 20;
    stack[2] = 30;
    stack[3] = 40;
    stack[4] = 50;

    printf("\n\nAfter pushing five elements:");
    isEmpty();
    isFull();
    getch();
}
```

```
Checking if stack is empty initially:
Stack is empty.
Checking if stack is full initially:
Stack is not full.

After pushing five elements:
Stack is not empty.
Stack is full.

...Program finished with exit code 255
Press ENTER to exit console.
```

Operator Precedence Rule

Operator	Precedence
Parentheses ()	Highest
Exponents ^	High
Multiplication *	Medium
Division /	Medium
Addition +	Low
Subtraction -	Low

Infix Expression/ Notation

- **Infix expressions** are mathematical expressions where the **operator is placed between its operands**.
- This is the most common mathematical notation used by humans. For example, the expression "2 + 3" is an infix expression, where the operator "+" is placed between the operands "2" and "3".
- **Infix notation** is easy to read and understand for humans, but it can be difficult for computers to evaluate efficiently.

Prefix Expression (Polish Notation)

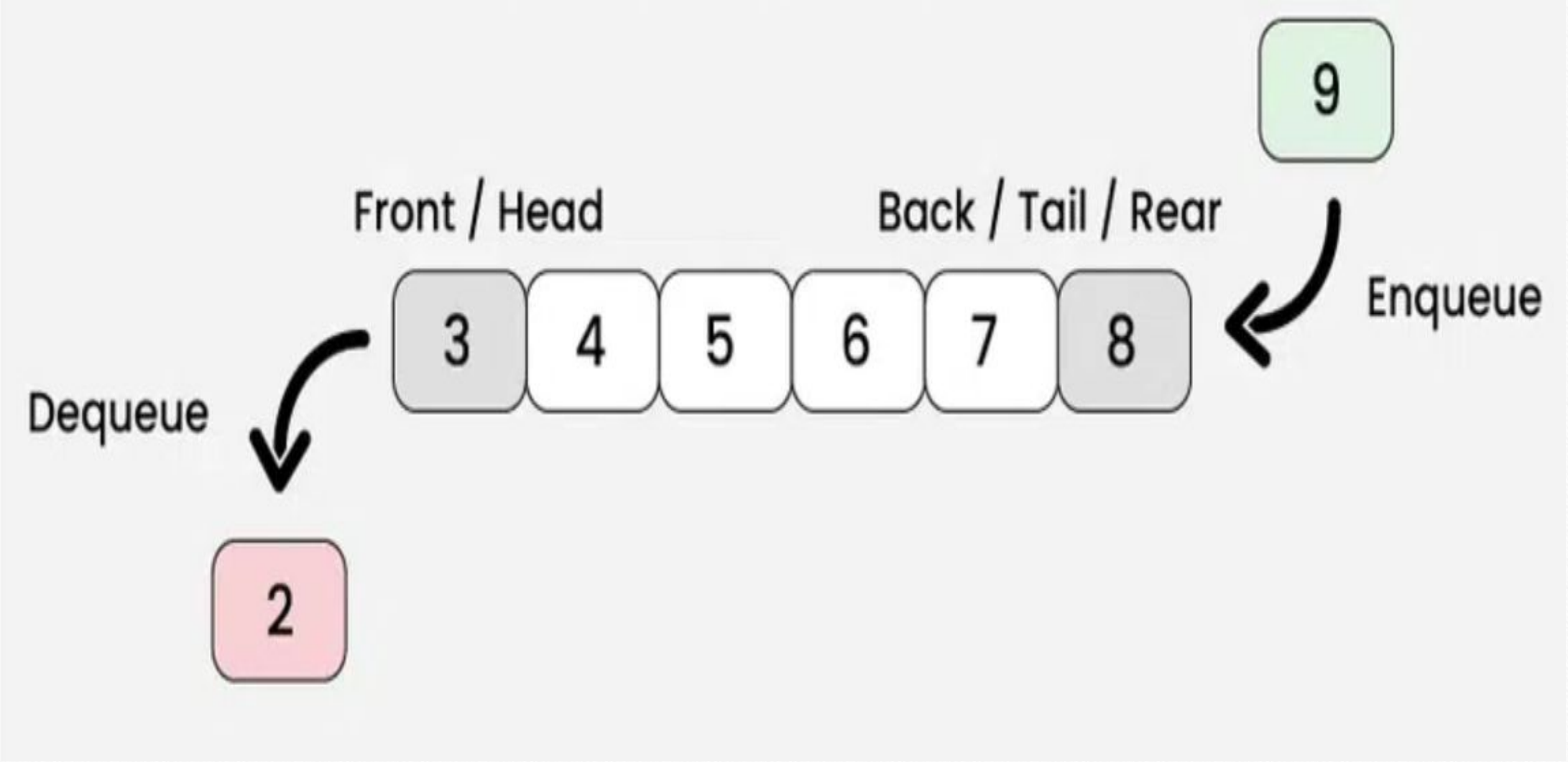
- **Prefix expressions** are also known as **Polish notation**, are a mathematical notation where the operator precedes its operands.
- In prefix notation, the operator is written first, followed by its operands. For example, the infix expression "a + b" would be written as "+ a b" in prefix notation.

Postfix Expression(Reverse Polish Notation)

- **Postfix expressions** are also known as **Reverse Polish Notation (RPN)**, are a mathematical notation where the **operator follows its operands**.
- In postfix notation, operands are written first, followed by the operator. For example, the infix expression "5 + 2" would be written as "5 2 +" in postfix notation.

QUEUE

- A linear data structure in which insertion can be done from one end(rear end) and deletion is done from other end(front end).
- It follows the principle of "**First in, First out" (FIFO)**, where the first element added to the queue is the first one to be removed.
- So, Queue is like a line of people waiting to purchase tickets, where the first person in line is the first person served. (i.e. First Come First Serve).


9
Front / Head
Back / Tail / Rear
Enqueue
3
4
5
6
7
8
Dequeue
2

Basic Operations

1. **Enqueue**: Adds an element to the end (rear) of the queue. If the queue is full, an overflow error occurs.
2. **Dequeue**: Removes the element from the front of the queue. If the queue is empty, an underflow error occurs.
3. **Peek/Front**: Returns the element at the front without removing it.
4. **Size**: Returns the number of elements in the queue.
5. **isEmpty**: Returns true if the queue is empty, otherwise false.
6. **isFull**: Returns true if the queue is full, otherwise false.
7. **rear**: returns the element at the rear end without removing it.

Types of Queue

- Linear Queue
- Circular Queue
- Double Ended Queue
- Priority Queue

Linear Queue

- In linear queue, insertion takes place at the rear end and removal occurs at the front.
- It strictly follows the FIFI (First In First Out) rule.

Circular Queue

- In a circular queue, The last element points to the first element making a circular link.
- The main advantage of a circular queue over simple queue is better memory utilization.
- If the last position is full and the first position is empty, we can insert an element in the first position .
- This action is not possible in a simple queue.

Double Ended Queue

- Queue in which insertion and deletion both can be performed from both ends.

Array Implementation of Queue

```
#include <stdio.h>
#include <stdlib.h>

struct Queue {
    int *arr;
    int front;
    int rear;
    int capacity;
};

struct Queue* createQueue(int capacity) {
    struct Queue* queue = (struct Queue*)malloc(sizeof(struct Queue));
    queue->capacity = capacity;
    queue->front = 0;
    queue->rear = -1;
    queue->arr = (int*)malloc(capacity * sizeof(int));
    return queue;
}

int isEmpty(struct Queue* queue) {
    return queue->front > queue->rear;
}
```

```c
void enqueue(struct Queue* queue, int x) {
    if (queue->rear < queue->capacity - 1) {
        queue->arr[++queue->rear] = x;
    }
}

void dequeue(struct Queue* queue) {
    if (!isEmpty(queue)) {
        queue->front++;
    }
}

int getFront(struct Queue* queue) {
    return isEmpty(queue) ? -1 : queue->arr[queue->front];
}

void display(struct Queue* queue) {
    for (int i = queue->front; i <= queue->rear; i++) {
        printf("%d ", queue->arr[i]);
    }
    printf("\n");
}
```

```c
int main() {
    struct Queue* q = createQueue(100);
    enqueue(q, 1);
    enqueue(q, 2);
    enqueue(q, 3);
    printf("%d\n", getFront(q));
    dequeue(q);
    enqueue(q, 4);
    display(q);
    return 0;
}
```

```
1
2 3 4



...Program finished with exit code 0
Press ENTER to exit console.
```

Linked List Implementation

```
#include <stdio.h>
#include <stdlib.h>

// Node structure definition
struct Node {
    int data;
    struct Node* next;
};

// Queue structure definition
struct Queue {
    struct Node* front;
    struct Node* rear;
};

// Function to create a new node
struct Node* newNode(int new_data) {
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));
    node->data = new_data;
    node->next = NULL;
    return node;
}
```

```c
// Function to initialize the queue
struct Queue* createQueue() {
    struct Queue* q = (struct Queue*)malloc(sizeof(struct Queue));
    q->front = q->rear = NULL;
    return q;
}

// Function to check if the queue is empty
int isEmpty(struct Queue* q) {
    return q->front == NULL;
}

// Function to add an element to the queue
void enqueue(struct Queue* q, int new_data) {
    struct Node* new_node = newNode(new_data);
    if (isEmpty(q)) {
        q->front = q->rear = new_node;
        printQueue(q);
        return;
    }
    q->rear->next = new_node;
    q->rear = new_node;
    printQueue(q);
}
```

```c
// Function to remove an element from the queue
void dequeue(struct Queue* q) {
    if (isEmpty(q)) {
        return;
    }
    struct Node* temp = q->front;
    q->front = q->front->next;
    if (q->front == NULL) q->rear = NULL;
    free(temp);
    printQueue(q);
}

// Function to print the current state of the queue
void printQueue(struct Queue* q) {
    if (isEmpty(q)) {
        printf("Queue is empty\n");
        return;
    }
    struct Node* temp = q->front;
    printf("Current Queue: ");
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}
```

```c
int main() {
    struct Queue* q = createQueue();

    // Enqueue elements into the queue
    enqueue(q, 10);
    enqueue(q, 20);

    // Dequeue elements from the queue
    dequeue(q);
    dequeue(q);

    // Enqueue more elements into the queue
    enqueue(q, 30);
    enqueue(q, 40);
    enqueue(q, 50);

    // Dequeue an element from the queue (this should print 30)
    dequeue(q);

    return 0;
}
```

```
Current Queue: 10
Current Queue: 10 20
Current Queue: 20
Queue is empty
Current Queue: 30
Current Queue: 30 40
Current Queue: 30 40 50
Current Queue: 40 50
```